

Program Analysis

– Project Description, Winter Semester 2024/25 –

Prof. Dr. Michael Pradel, Aryaz Eghbali, Huimin Hu

November 18, 2024

1 Introduction

Program slicing is a type of program analysis that extracts a subprogram from the source code to facilitate downstream tasks such as locating the root cause of a bug, or understanding the effects of a code change. There are multiple slicing algorithms, some of which are discussed during the lecture.

2 Project Description

The goal of this project is to implement *dynamic forward slicing* for Python. The slicing process starts from a specific line of code, called the *slicing criterion*, and extracts the subset of the program that is affected by modifying the slicing criterion. We call the resulting subprogram the *forward slice* w.r.t. the slicing criterion. The project computes the slice dynamically, i.e., based on a concrete execution of the original program.

The exact choice of the slicing algorithm is up to you, and in particular, you are not restricted to algorithms discussed in the lecture. The project will be based on DynaPyt¹, a general purpose framework for dynamically analyzing Python. Your implementation should also be in Python. Given a Python program and a slicing criterion, the final output of your analysis should produce a subprogram that contains only the code in the slice, e.g., by removing subtrees from the AST that are not part of the slice. To reduce the given source code into the slice, libraries like LibCST² will be helpful for parsing, manipulating, and generating code.

2.1 Scope and Limitations

To help students focus on the main analysis and avoid the hassle of dealing with all of the language features of Python, we narrow down the scope of programs that your implementation should be able to handle:

- You will implement an intra-procedural analysis, which means your analysis is limited to the execution of a function. Specifically, the approach should only slice the body of the function named `slice_me` and must not modify any code outside of its body. That is, the function header (function name, parameters, and any type hints) must remain intact.
- You can assume that any other functions called during the execution of the `slice_me` function have the following side-effects:
 - There is a data flow from any argument to the return value. E.g., `z = foo(x)` introduces a data flow from `x` to `z`.
 - If the call is a method call, then there is a data flow from the object on which the method is called to the return value, from the arguments to the object on which the method is called, and from the arguments to the return value. E.g., `z = y.foo(x)` introduces data flows `y → z`, `x → y`, and `x → z`.

¹<https://github.com/sola-st/DynaPyt>

²<https://github.com/Instagram/LibCST>

- There are no other side-effects, i.e., a called function is assumed to not change the values of any non-local variables or objects.
- Calls to `exec` and `eval`, and `with` statements are considered out of the scope of this project, and will not appear in our test cases.
- The left-hand side of every assignment (or augmented assignment) is either a variable (`foo=`), an attribute access (`foo.bar=`), or an index access (`foo[bar]=`).
- Your analysis should be able to handle complex objects and arrays. You can assume that any modifications to the object or array causes a dataflow edge to any read of attributes or any index accesses of the object or array. For example, modifying `a[i]` is assumed to cause a dataflow edge to any read of `a[j]`.
- There are no definitions of other functions or classes inside the analyzed function.
- If the slice has a compound statement (e.g., an `if` statement), where all of its bodies are empty (e.g., the `if` itself and all of its `elifs` and `else`), then the compound statement should be removed from the slice. If however, at least one of the bodies is not empty, then the compound statement should be kept in the slice, but the empty bodies should be replaced with `pass`.

3 Milestones

The project is broken down into three milestones for better guidance throughout the semester. Each milestone has a progress meeting, in which each student meets with her/his mentor, presents the deliverable, and seeks help if needed. Please create a **private** GitHub repository for your project and give read access to your mentor. Some code templates have been prepared for this project, on top of which you should implement your solution. The template is provided as a Python package name `dynamicslicing`. Please adhere to the output format shown in the provided test cases, as the output of your program will be automatically tested.

3.1 Milestone 1 (Progress Meeting in Week of Nov 25–29, 2024)

In the first part of the project you will familiarize yourself with DynaPyt by implementing a simple dynamic analysis. Furthermore, you will also implement utility methods that are required for manipulating the source code and for printing the final result. In the progress meeting of this milestone, course participants are expected to have implemented the following:

1. A dynamic analysis in DynaPyt that each time a variable is being written to, prints the value being written. This analysis must be implemented in `dynamicslicing/trace_writes.py`. For example, in the following program

```

1 def slice_me():
2     x = 5
3     print("Hello World")
4     if x < 10:
5         x += 5
6     y = 0
7     return y
8
9 slice_me()
```

the output (including both analysis output and program output) should be

```

5
Hello World
10
0
```

As a starting point, we suggest working through this tutorial.

2. A function that, given the path of a Python file and a set of code lines to keep, removes nodes from the syntax tree that are not on these lines. To this end, the function should parse the code, e.g., with LibCST, manipulate the syntax tree, and then return the new code. This function should be implemented in `dynamicslicing/Utils.py` as the `remove_lines` function. For example, given the above source code and lines `[1, 2, 4, 5, 9]`, the function should produce

```
1 def slice_me():
2     x = 5
3     if x < 10:
4         x += 5
5
6 slice_me()
```

Note that the correctness evaluation is based on LibCST's `deep_equals` on the root of the syntax trees. For your reference, there is a sample code that uses LibCST to negate the condition of all `if`-statements on odd lines. This sample is implemented in `dynamicslicing/Utils.py` in the `OddIfNegation` class and the `negate_odd_ifs` function. There is a naive solution to this part of Milestone 1, which is to remove the lines directly from the string representation of the code. However, this naive approach can result in syntax errors in the resulting code, which is why you should implement a syntax tree-based solution.

3.2 Milestone 2 (Progress Meeting in Week of Dec 16–20, 2024)

The expectation for this milestone is that each student has implemented dynamic program slicing based on data-flow dependencies. This analysis must be implemented in `dynamicslicing/slice_dataflow.py` as the `SliceDataflow` class. For the following example,

```
1 def slice_me():
2     x = 1 # slicing criterion
3     y = 2
4     x = x + y
5     y += 2
6     return y
7
8 slice_me()
```

the correct output is

```
1 def slice_me():
2     x = 1 # slicing criterion
3     x = x + y
4
5 slice_me()
```

This output should be written to a file named `sliced.py` in the same directory as the input file. Note that you must not change the signature of the constructor of `SliceDataflow`.

Each input for this milestone consists of an input Python file, which is the program to be sliced, containing a comment, which gives the slicing criterion. You can safely assume that, in this milestone, the input code consists only of functions with a single control flow path, i.e., no branches or loops.

You can test your code using `pytest tests --only tests/milestone2`. You can, and are strongly encouraged to, add your own test cases to the `tests/milestone2` directory. Each test case consists of one program file and one expected output file, bundled in a directory, similar to the provided test cases.

3.3 Milestone 3 (Progress Meeting in Week of Jan 20–24, 2025)

It is expected that each course participant has implemented the complete slicing algorithm, including both data-flow and control-flow dependencies. This analysis must be implemented in `dynamicslicing/slice.py` as the `Slice` class. For example, the output of your implementation for

```

1 def slice_me():
2     x = 1
3     y = 2 # slicing criterion
4     z = 3
5     if x < 4:
6         y += 2
7     if x > 0:
8         z -= 5
9     return y
10
11 slice_me()

```

should be

```

1 def slice_me():
2     y = 2 # slicing criterion
3     y += 2
4     return y
5
6 slice_me()

```

This output should be written to a file named `sliced.py` in the same directory as the input file. Similar to the second milestone, the input of your analysis is a Python file containing a comment, which indicate the program to be sliced and the slicing criterion, respectively. Note that similar to the previous milestone, you can test your analysis with Pytest, and you must not change the signature of the constructor of the analysis class.

4 Installation Guidelines

Please refer to DynaPyt's tutorial and documentation for installation instructions. To install your analysis as a package, run the following from the root directory of the project:

```

pip install -r requirements.txt
pip install -e .

```

5 Running and Testing Your Code

5.1 Implementation and Execution

You should implement your analyses in the appropriate files described above, and must not change any function signature from the provided skeleton code. Your implementation for all milestones should generate a file named `sliced.py`, located in the same directory as the provided input file. You can use Python versions between 3.9 and 3.13 (inclusive) for your implementation. We suggest using a containerized environment (e.g., Docker) or a virtual environment to install the dependencies and run your code.

5.2 Testing

For the first milestone, you are supposed to run DynaPyt's instrumentation script and analysis using its command line interface (CLI). For the second and third milestones, you can use Pytest to run your analysis on the test cases provided in the `tests` directory.

To run the test cases in the `tests` directory, you can use the following command:

```
pytest tests
```

To run the tests for one of the milestones only, run:

```
pytest tests --only tests/milestoneX
```

where `milestoneX` can be `milestone2` or `milestone3`. In addition to the provided test cases, we strongly recommend to test your code with additional test cases. To add test cases, you should create an input file, named `program.py`, and an expected output file, named `expected.py`. Your implementation will be evaluated with test cases that are not provided to you.

For consistency and to ease the task, all input code files follow this convention:

- There is a single function definition, with the function called `slice_me`, which is the function to be sliced.
- The last statement in the file is a call to the `slice_me` function.

The examples given above follow this convention.

6 Mentoring and How to Submit

Each student has a mentor, either Aryaz Eghbali (aryaz.eghbali@iste.uni-stuttgart.de) or Huimin Hu (huimin.hu@iste.uni-stuttgart.de). Students must meet their mentor at least three times during the semester, on the dates indicated for the milestones. In addition, students may consult their mentor to resolve any questions, to ensure progress in the right direction, and to help you submit a successful project in time. We also recommend to ask general questions in the Ilias forum.

The deadline for submitting the project is at 11:59 PM on February 5, 2025. This deadline is firm. The submission will be via Ilias and should include:

- A scientific report of at most four pages that summarizes the approach taken, the results obtained, and any shortcomings you are aware of.
- A .zip file of the `dynamicslicing` directory (the top directory which includes `src`, `tests`, etc.) with your implementation. Your implementation will be evaluated automatically, by installing it and running Pytest. Please do not change the interface of the analysis classes.

Each person must present the project on the week of February 10–14, 2025, in a short talk, followed by a question and answer session.

The project is individual, i.e., students must work by themselves and not share their solution with anyone else. Any form of collaboration that results in similar or identical solutions being submitted will be punished according to the usual rules for plagiarism.

Grading will be based on the following criteria:

Criterion	Contribution
Progress meetings and final presentation (progress made, clarity, illustration, quality of answers)	25%
Implementation (completeness, documentation)	25%
Results (soundness, reproducibility)	25%
Report (clarity, illustration, discussion and interpretation of the results)	25%